

parcial 1a		parcial 2a		final	X
------------	--	------------	--	-------	---

MATERIA:	CERTIFICACIÓN
DOCENTE:	ING. HERNAN PAYRUMANI

Puntaje total de la prueba:	100	puntos	Fecha:	16-09-2026
-----------------------------	-----	--------	--------	------------

DATOS QUE DEBE COMPLETAR EL ESTUDIANTE:

Código:		Nombre :	
---------	--	----------	--

El proyecto consiste en diseñar, implementar, explicar y publicar un juego web. La aplicación tendrá un frontend desarrollado con React y un backend desarrollado con Express.

El objetivo principal no es producir la mayor cantidad de código. Se espera que el estudiante comprenda el flujo completo: una acción modifica el estado del juego, React presenta el resultado, el frontend se comunica con Express y el servidor conserva o entrega información.

Se permite utilizar agentes de inteligencia artificial como asistentes. Sin embargo, cada estudiante debe poder explicar, probar y modificar el código que presenta.

Elección libre del juego

Cada estudiante investigará y propondrá el juego que desea desarrollar. La elección es abierta, pero debe cumplir todos los lineamientos mínimos de esta propuesta. No habrá aprobación previa de la idea.

No se aceptan tres en raya, buscaminas, adivinanzas basadas en tablas, Space Invaders ni juegos iguales o parecidos a los desarrollados en clase. El juego debe ser una idea original pensada por el estudiante y no una modificación de un ejercicio existente.

La propuesta debe corresponder a las herramientas y requisitos definidos en este documento. El estudiante debe poder explicar las reglas, la arquitectura, las pruebas y el funcionamiento de su implementación.

Lineamientos mínimos del juego

El juego elegido debe cumplir todos los requisitos siguientes:

1. **Al menos dos jugadores.** Pueden jugar en el mismo dispositivo, desde dispositivos distintos o contra una estrategia controlada por el backend.
2. **Uso significativo de la pantalla.** El juego debe utilizar toda el área visible del navegador como espacio de juego, no únicamente una porción limitada o un formulario centrado. Debe integrar un escenario o tablero, información de jugadores, marcador, controles y estado de la partida.

parcial 1a		parcial 2a		final	X
------------	--	------------	--	-------	---

MATERIA:	CERTIFICACIÓN
DOCENTE:	ING. HERNAN PAYRUMANI

3. **Elementos en movimiento.** Algunos elementos deben cambiar de posición, desplazarse, aparecer, desaparecer o interactuar durante la partida, por tiempo, turnos o acciones.
4. **Interacción entre jugadores.** Los jugadores deben afectar un estado compartido mediante ataques, defensa, captura, bloqueo, competencia por recursos, negociación u otra dinámica equivalente.
5. **Estado no trivial.** El juego debe relacionar al menos tres tipos de estado, como posición, vida, energía, recursos, inventario, turnos, tiempo, objetivos o puntuación.
6. **Reglas y finalización.** Deben definirse el inicio, las acciones válidas e inválidas, los turnos o el tiempo, la victoria, el empate y la finalización.
7. **Decisión estratégica.** El jugador debe elegir entre alternativas con consecuencias diferentes. No se aceptará un funcionamiento basado únicamente en presionar repetidamente el mismo control.
8. **Variabilidad.** Cada partida debe presentar alguna diferencia mediante eventos, objetivos, mapas, recursos, cartas, misiones o comportamiento variable del adversario.
9. **Retroalimentación visual.** El usuario debe comprender el estado de la partida sin abrir la consola. Los turnos, recursos, errores, resultados y cambios importantes deben ser visibles. Uso de imágenes como imágenes pixeladas o normales.

No se aceptan juegos ya desarrollados en clase, una colección de minijuegos independientes, una aplicación que solo registre puntajes ni un backend que no participe en la partida. Tampoco se permite utilizar una librería externa de juegos para resolver las reglas principales.

Documentación de decisiones del proyecto

No habrá una etapa de aprobación previa. Cada estudiante debe documentar sus decisiones de diseño y planificación en uno o más archivos Markdown dentro de la carpeta **docs/** del repositorio. Esta documentación debe existir antes de la entrega final y mostrar cómo evolucionó la idea durante el desarrollo.

Como mínimo, la documentación debe incluir:

- nombre del juego y descripción de cómo se juega. Debe explicar qué observa y realiza el usuario, qué decisiones toma, cómo responde el sistema y cómo termina la partida;
- introducción al proyecto, su propósito y la experiencia de juego que propone;
- cantidad y tipo de jugadores;
- reglas y condición de victoria;
- elementos que se mueven;
- estados principales e interacción entre jugadores;
- responsabilidades de React y de Express;

parcial 1a		parcial 2a		final	X
------------	--	------------	--	-------	---

MATERIA:	CERTIFICACIÓN
DOCENTE:	ING. HERNAN PAYRUMANI

- diseño de la API HTTP REST, con endpoints, método, entrada, salida y ejemplos de solicitudes y respuestas JSON;
- boceto de la pantalla;
- decisiones técnicas y su justificación;
- riesgos técnicos y estrategia para reducirlos;
- cambios importantes realizados durante el desarrollo y su justificación;
- investigación y configuración de las pruebas E2E y del servicio de publicación. Si utiliza Docker, debe documentar también esa decisión.

Una organización posible es [docs/introduccion.md](#), [docs/reglas.md](#), [docs/api.md](#), [docs/decisiones.md](#) y [docs/investigacion.md](#). Los nombres pueden variar, pero los archivos deben estar escritos en Markdown, formar parte del repositorio y ser suficientemente claros para comprender el proyecto sin revisar todo el código.

Requisitos técnicos obligatorios

- El frontend debe estar implementado con React y TypeScript. El backend debe estar implementado con Express y TypeScript.
- No se deben utilizar librerías externas para resolver la lógica principal, la comunicación, el estado, el routing o la interfaz. Se permiten React, Express, TypeScript, las herramientas mínimas de construcción y una herramienta de pruebas E2E. No se permite Bootstrap, Tailwind, Axios, React Router, Redux, motores de juegos ni bibliotecas de componentes.
- React debe comunicarse realmente con Express mediante solicitudes HTTP usando la API nativa `fetch`. No se aceptará una conexión simulada ni datos exclusivamente escritos en el frontend.
- Toda la comunicación de la API debe utilizar JSON tanto para la entrada como para la salida. El juego debe tener, como mínimo, un endpoint `GET` y un endpoint `POST` relacionados con la partida.
- El frontend y el backend deben funcionar bajo el mismo dominio y puerto. Express debe servir el frontend compilado o utilizar una configuración equivalente que permita acceder a toda la aplicación desde una única dirección.
- El proyecto debe incluir estilos propios en CSS y una interfaz usable en el navegador.
- El repositorio debe tener al menos tres GitHub Actions o trabajos claramente diferenciados: validación mediante linters, ejecución de pruebas E2E y deployment de la aplicación completa a un servidor. Cada workflow debe indicar su propósito y producir un resultado verificable.
- La validación mediante linters debe cubrir frontend y backend, y debe fallar si el código no cumple las reglas configuradas.

parcial 1a		parcial 2a		final	X
------------	--	------------	--	-------	---

MATERIA:	CERTIFICACIÓN
DOCENTE:	ING. HERNAN PAYRUMANI

- Se deben implementar pruebas end-to-end con Playwright, Cypress u otra herramienta equivalente. Las pruebas deben cubrir el inicio, la interacción principal, la comunicación con el backend y un caso de finalización o validación.
- Las pruebas E2E deben ejecutarse automáticamente en GitHub Actions en modo headless y localmente durante la defensa en Chrome de forma visual. La misma prueba o un conjunto equivalente debe demostrar el comportamiento de la aplicación publicada.
- La aplicación completa, incluyendo frontend y backend, debe desplegarse en un servicio accesible mediante una URL. El servicio y la estrategia de publicación quedan a elección del estudiante. Docker es opcional y dependerá del servicio elegido. Se recomienda investigar [Render/https://render.com/](https://render.com/). Si utiliza Docker, debe entregar el **Dockerfile**, explicar el puerto, las variables necesarias y el comando de inicio.
- El repositorio debe contener instrucciones suficientes para instalar, ejecutar, probar y desplegar el proyecto.

Alcance funcional mínimo

El juego debe incluir una pantalla de inicio, una partida jugable, instrucciones visibles, mensajes para estados importantes y una pantalla o sección de resultado. Debe contemplar al menos una acción inválida o un caso límite, como un intento repetido, un turno incorrecto, un formulario vacío o una partida terminada.

El backend debe participar en una decisión significativa. Por ejemplo, puede generar el estado inicial, validar una jugada, calcular el resultado, registrar el puntaje o devolver un historial. Guardar información únicamente en una variable del navegador no demuestra integración suficiente.

Responsabilidades de React y Express

React debe representar el escenario, recibir las interacciones, mostrar el estado y actualizar la interfaz. Express debe asumir al menos dos responsabilidades relevantes de la partida, como administrar jugadores, crear la partida, validar acciones, procesar turnos, generar elementos, calcular resultados, guardar el estado o entregar un historial. La lógica crítica no debe existir únicamente en el navegador.

Investigación técnica

Los requisitos de pruebas E2E y publicación constituyen también actividades de investigación. El estudiante debe explicar en la documentación de **docs/** qué herramienta investigó, qué fuentes consultó, cómo ejecutó las pruebas localmente y qué limitaciones encontró. También debe documentar el proceso de publicación, la configuración del puerto, las variables de entorno y las características del

parcial 1a		parcial 2a		final	X
------------	--	------------	--	-------	---

MATERIA:	CERTIFICACIÓN
DOCENTE:	ING. HERNAN PAYRUMANI

servicio elegido. La investigación y el uso de Docker son opcionales y dependen de la estrategia de despliegue seleccionada.

Entregables

El repositorio y la aplicación publicada son los entregables principales. Ambos deben existir, estar accesibles y corresponder al mismo trabajo. Si no existe el repositorio o no existe una aplicación publicada mediante una URL funcional, el estudiante no podrá acceder a la defensa del examen y no se procederá con la calificación por puntos.

Como requisito adicional para ingresar al examen y a la defensa, el repositorio de GitHub debe estar actualizado como máximo hasta el **15 de septiembre de 2026 a horas 16:00**, tomando como referencia la fecha y hora registrada por GitHub. Cualquier cambio, commit o actualización posterior a ese plazo anulará el proyecto y no se asignará nota.

La entrega incluye:

- repositorio GitHub accesible y cerrado para evaluación, con historial de commits comprensible y todos los archivos del proyecto;
- aplicación publicada, con una URL funcional para acceder al juego;
- **README.md** con requisitos, comandos, arquitectura, endpoints JSON, variables de entorno y enlace al despliegue. Si utiliza Docker, también debe incluir sus instrucciones;
- carpeta **docs/** con los archivos Markdown que documentan el proyecto, sus decisiones y la investigación técnica;
- archivos de workflow de GitHub Actions, linters de frontend y backend, y evidencia de ejecuciones exitosas;
- código de las pruebas E2E y comando para ejecutarlas localmente en Chrome de forma visual. La ejecución headless debe estar configurada en GitHub Actions;
- si utiliza Docker, **Dockerfile** y evidencia de construcción y ejecución local de la imagen;
- video de entre 3 y 5 minutos mostrando una partida, una solicitud JSON, una prueba E2E en Chrome, el resultado de las GitHub Actions y la aplicación completa publicada;
- documentación completa en archivos Markdown dentro de **docs/**, incluyendo el registro del uso de IA;
- defensa individual y modificación solicitada por el docente.

parcial 1a		parcial 2a		final	X
------------	--	------------	--	-------	---

MATERIA:	CERTIFICACIÓN
DOCENTE:	ING. HERNAN PAYRUMANI

Uso responsable de agentes de IA

La IA puede utilizarse para proponer código, explicar errores, generar casos de prueba o sugerir mejoras. La documentación en [docs/](#) debe registrar las solicitudes relevantes, indicar qué respuestas fueron incorporadas y describir qué verificó el estudiante. No se penaliza utilizar IA. Se penaliza presentar código que el estudiante no puede explicar o modificar.

Sesión de defensa

La defensa será individual. El estudiante ingresará al aula y dispondrá de un máximo de 10 minutos, controlados mediante un cronómetro. El tiempo comienza cuando inicia su presentación. La preparación del material y la apertura de las herramientas deben estar resueltas antes de comenzar.

Durante la sesión se realizarán dos pruebas obligatorias:

1. **Ejecución E2E en producción.** El estudiante ejecutará una prueba end-to-end mediante Playwright u otra herramienta equivalente, utilizando Chrome de forma visual y la URL pública de la aplicación. Debe mostrar que la prueba interactúa con el juego publicado y verifica una parte relevante de la partida.
2. **Cambio solicitado e integración continua.** El docente solicitará un cambio concreto. El estudiante deberá realizarlo, ejecutar o activar las GitHub Actions y demostrar la validación mediante linters, la ejecución E2E en ambiente headless y el deployment de la aplicación completa. Finalmente, el cambio debe verse reflejado en la página web publicada.

El estudiante debe preparar con anticipación el flujo de publicación automática o el procedimiento necesario para que el cambio solicitado pueda llegar a la aplicación publicada durante la defensa. No se considerará suficiente mostrar el cambio únicamente en el entorno local.

Una vez concluidas satisfactoriamente estas dos pruebas, se procederá a revisar los puntos de la rúbrica. Si el estudiante no termina las pruebas obligatorias dentro de los 10 minutos, la defensa se considerará inválida y no se asignará nota por puntos.

Es responsabilidad del estudiante llevar el material necesario para la defensa, incluyendo una laptop con batería suficiente, cargador cuando corresponda, acceso al repositorio, credenciales disponibles, conexión o recursos necesarios y cualquier archivo requerido para ejecutar las pruebas. No se ampliará el tiempo por falta de preparación del material.

parcial 1a		parcial 2a		final	X
------------	--	------------	--	-------	---

MATERIA:	CERTIFICACIÓN
DOCENTE:	ING. HERNAN PAYRUMANI

Rúbrica de evaluación

La originalidad del juego no es un criterio que otorgue puntos, sino un requisito de validez. Si el juego es igual o parecido a uno de los juegos realizados en clase, el proyecto será inválido y no se asignará nota.

La calificación se organiza en 10 secciones de 10 puntos cada una, para un total de 100 puntos. Esta rúbrica se aplica únicamente después de verificar los requisitos habilitantes de acceso a la defensa.

Sección	Puntos	Evidencia
Reglas y experiencia jugable	10	Reglas claras, decisiones, estados y condición de finalización.
Interacción entre jugadores y movimiento	10	Dos jugadores, interacción significativa y elementos en movimiento.
Implementación con React y TypeScript	10	Componentes, estado, eventos y renderizado comprensible.
Backend con Express y TypeScript	10	Rutas, lógica relevante del servidor y respuestas JSON.
Integración HTTP REST	10	Entrada y salida JSON, uso real de fetch , mismo dominio y puerto.
GitHub Actions y linting	10	Al menos tres Actions: linting, E2E y deployment, con validaciones verificables.
Pruebas E2E	10	Ejecución headless en GitHub Actions y ejecución visual en Chrome.
Publicación de la aplicación	10	Frontend y backend completos accesibles mediante una URL pública.
Repositorio y documentación técnica	10	Repositorio accesible, README y archivos Markdown en docs/ .
Defensa individual y modificación	10	Explicación, razonamiento y cambio solicitado.